



UNIVERSITY OF ESWATINI

UNIVERSITY OF ESWATINI

GROUP MEMEBERS:

1. **LENHLE DLAMINI: 202302047**
2. **GCINWAYINKHOSI BHEMBHE: 202405187**
3. **NSIKA MKHATSHWA: 202401362**
4. **ANDISWA MDLULI: 202304513**
5. **LINDOKUHLE NKABINDE: 202302158**

Smart hostel innovators

Project title: Hostel Access Management System

Software Design Document (SDD)

Names:

1. Gcinwayinkhosi Bhembe :202405187
2. Lenhle Dlamini: 202302047
3. Nsika Mkhathshwa: 202401362
4. Lindokuhle Nkabinde: 202302158
5. Andiswa Mdluli: 202304513

Department: Computer science

lab section : MFAC ROOM 1

University of Eswatini

Date:

12 May 2026

TABLE OF CONTENTS

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Overview
 - 1.4 Reference Material
 - 1.5 Definitions and Acronyms
2. System Overview
3. System Architecture
 - 3.1 Architectural Design
 - 3.2 Decomposition Description
 - 3.3 Design Rationale
4. Data Design
 - 4.1 Data Description
 - 4.2 Data Dictionary
5. Component Design
6. Human Interface Design
 - 6.1 Overview of User Interface
 - 6.2 Screen Images
 - 6.3 Screen Objects and Actions
7. Requirements Matrix
8. Appendices

1. INTRODUCTION

1.1 Purpose

The purpose of this Software Design Document (SDD) is to describe the overall architecture and design of the Hostel Access Management System. The document provides a detailed explanation of the software components, interfaces, database structures, and system behavior required for implementation. This document is intended for developers, testers, lecturers, and project stakeholders.

1.2 Scope

The Hostel Access Management System is designed to improve security and management of student hostels within the university campus. The system automates hostel operations such as student registration, room allocation, hostel access verification, and report generation.

The system will help:

- Reduce unauthorized access
- Improve hostel security
- Simplify room allocation
- Maintain accurate hostel records
- Improve monitoring of student movement
- Reduce paperwork and manual operations

The system users include:

- Students
- Hostel administrators
- Security personnel

1.3 Overview

This document describes the software architecture, data design, component design, user interface design, and requirements mapping of the Hostel Access Management System. It also explains how the different system modules interact to achieve the required functionality.

1.4 Reference Material

The following materials were used during development of this document:

- IEEE Std 1016-2009 Software Design Description Standard
- Software Engineering lecture notes
- Database Systems course materials
- Hostel Management System references
- Software Requirements Specification (SRS)

1.5 Definitions and Acronyms

Term	Meaning
SDD	Software Design Document
SRS	Software Requirements Specification
RFID	Radio Frequency Identification
DBMS	Database Management System
Admin	Hostel Administrator
User	Any person using the system
Access Log	Record of hostel entry or exit
Authentication	Process of verifying user identity

2. SYSTEM OVERVIEW

The Hostel Access Management System is a web-based system developed to improve management and security of student hostels within the university campus. The system allows administrators to manage student records, assign hostel rooms, monitor hostel access, and generate reports.

Students will use login credentials or RFID-enabled student cards to access hostel facilities. Security personnel will be able to monitor hostel entry and exit activities in real time.

The system consists of the following major modules:

- User Authentication Module
- Student Management Module
- Room Allocation Module
- Hostel Access Control Module
- Reporting Module
- Database Management Module

The system is expected to improve efficiency, security, and record management within university hostels.

3. SYSTEM ARCHITECTURE

3.1 Architectural Design

The system uses a three-layer architecture consisting of:

1. Presentation Layer
2. Application Layer
3. Database Layer

Presentation Layer

This layer contains the graphical user interface used by students, administrators, and security personnel.

Application Layer

This layer handles business logic such as authentication, room allocation, and access verification.

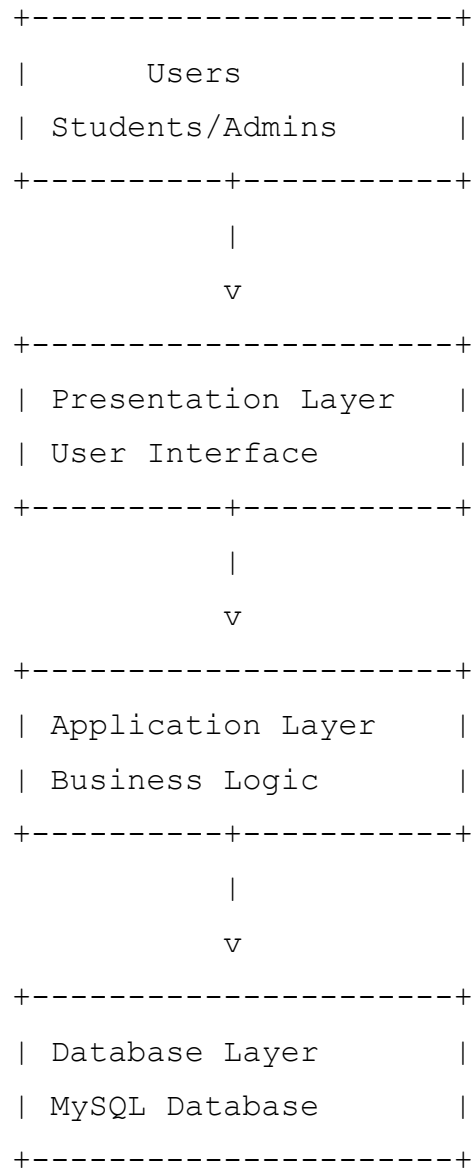
Database Layer

This layer stores all system data including student records, hostel information, access logs, and room allocations.

Major Subsystems

Subsystem	Responsibility
Authentication Module	Verifies user login credentials
Student Management Module	Manages student information
Room Allocation Module	Allocates hostel rooms
Access Control Module	Controls hostel access
Reporting Module	Generates hostel reports
Database Module	Stores and manages system data

Architectural Diagram



3.2 Decomposition Description

The system follows an object-oriented design approach.

Main Classes

// Main Class

```
public class HostelAccessSystem {
```

```
    // Method to start the system
```

```
    public void startSystem() {
```

```
        System.out.println("Hostel Access Management System Started");
```

```
    }
```

```
    // Method for database connection
```

```
    public void connectDatabase() {
```

```
        System.out.println("Database Connected Successfully");
```

```
    }
```

```
    // Method to display login page
```

```
    public void displayLogin() {
```

```
        System.out.println("Displaying Login Page");
```

```
}
```

```
// Method to close the system
```

```
public void shutdownSystem() {
```

```
    System.out.println("System Shutdown");
```

```
}
```

```
// Main method
```

```
public static void main(String[] args) {
```

```
    HostelAccessSystem system = new HostelAccessSystem();
```

```
    system.startSystem();
```

```
    system.connectDatabase();
```

```
    system.displayLogin();
```

```
// Creating student object
```

```
Student student1 = new Student(
```

```
    "202405187",
```

```
        "Gcinwayinkhosi Bhembe",  
  
        "Information Science",  
  
        "S Hostel",  
  
        "S415"  
  
    );
```

```
// Display student information  
  
student1.viewStudentDetails();
```

```
// Student login  
  
student1.login();
```

```
// Student requests hostel access  
  
student1.requestAccess();
```

```
// Student views room allocation  
  
student1.viewAllocation();
```

```
system.shutdownSystem();
```

```
}
```

```
}
```

Student Class

```
// Student Class
```

```
public class Student {
```

```
    // Attributes
```

```
    private String studentID;
```

```
    private String fullName;
```

```
    private String programme;
```

```
    private String hostelName;
```

```
    private String roomNumber;
```

```
    // Constructor
```

```
    public Student(String studentID, String fullName,
```

```
                    String programme, String hostelName,
```

```
                    String roomNumber) {
```

```
        this.studentID = studentID;
```

```
this.fullName = fullName;

this.programme = programme;

this.hostelName = hostelName;

this.roomNumber = roomNumber;

}


// Method for login

public void login() {

    System.out.println(fullName + " logged into the system.");

}


// Method to request hostel access

public void requestAccess() {

    System.out.println(fullName + " requested hostel access.");

}


// Method to view room allocation

public void viewAllocation() {

    System.out.println("Hostel: " + hostelName);
```

```
        System.out.println("Room Number: " + roomNumber);

    }

    // Method to display student details

    public void viewStudentDetails() {

        System.out.println("\n===== STUDENT DETAILS =====");

        System.out.println("Student ID: " + studentID);

        System.out.println("Full Name: " + fullName);

        System.out.println("Programme: " + programme);

        System.out.println("Hostel: " + hostelName);

        System.out.println("Room Number: " + roomNumber);

    }

}
```

Attributes:

- Student ID
- Full Name
- gender
- programe

- room Number

Methods:

- login()
- requestAccess()
- viewAllocation()

Admin Class

Attributes:

- adminID
- username
- password

Methods:

- allocateRoom()
- registerStudent()
- generateReport()

Hostel Class

Attributes:

- hostelID
- hostelName
- capacity

Methods:

- addRoom()
- viewRooms()

AccessLog Class

Attributes:

- logID
- studentID
- accessTime
- accessStatus

Methods:

- recordEntry()
- viewLogs()

Use Case Diagram

```

Student -----> Login
Student -----> View Room Allocation
Student -----> Access Hostel

Admin -----> Register Student
Admin -----> Allocate Room
Admin -----> Generate Reports

Security ----> Monitor Access Logs
  
```

Sequence Diagram

Student -> Login Page -> Authentication Module
Authentication Module -> Database
Database -> Authentication Module
Authentication Module -> Student Dashboard

3.3 Design Rationale

The layered architecture was selected because it improves maintainability, scalability, and security. Separating the user interface from the business logic and database makes the system easier to manage and update.

The object-oriented design approach was selected because it supports modularity and code reuse. It also simplifies future system enhancements.

Alternative architectures such as monolithic architecture were considered but not selected because they are difficult to maintain as the system grows.

4. DATA DESIGN

4.1 Data Description

The system stores information using a relational database management system.

Main Entities

- Students
- Administrators
- Hostels
- Rooms
- Access Logs

Entity Relationships

- One hostel contains many rooms.
- One student occupies one room.
- One student can generate many access logs.

ERD Structure

Student ---- assigned to ---- Room
Room ---- belongs to ---- Hostel
Student ---- generates ---- AccessLog

4.2 Data Dictionary

Entity	Attribute	Type	Description
Student	studentID	Integer	Unique student identifier
Student	fullName	Varchar	Student full name
Student	programme	Varchar	Student programme

Entity	Attribute	Type	Description
Room	roomNumber	Integer	Assigned room number
Hostel	hostelName	Varchar	Name of hostel
AccessLog	accessTime	DateTime	Time of hostel entry
AccessLog	accessStatus	Varchar	Entry status
Admin	username	Varchar	Administrator username

5. COMPONENT DESIGN

Login Function

```
BEGIN
    INPUT username
    INPUT password

    IF username and password are correct THEN
        grant access
    ELSE
        display error message
    ENDIF
END
```

Student Registration Function

```
BEGIN
    INPUT student details

    SAVE student information into database

    DISPLAY registration successful
END
```

Room Allocation Function

```
BEGIN
    SELECT available room

    ASSIGN student to room
```

```
    UPDATE room records

    DISPLAY allocation successful
END
```

Access Verification Function

```
BEGIN
    SCAN student card

    VERIFY student details

    IF student exists THEN
        allow access
        record entry time
    ELSE
        deny access
    ENDIF
END
```

Report Generation Function

```
BEGIN
    SELECT report type

    RETRIEVE records from database

    GENERATE report

    DISPLAY report
```

END

6. HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

The system provides a simple and user-friendly interface that allows users to navigate easily through different system features.

Main Interfaces

- Login Page
- Student Dashboard
- Admin Dashboard
- Room Allocation Interface
- Access Monitoring Interface
- Report Generation Interface

Students can log in to view their room allocation and hostel status. Administrators can manage hostel records and generate reports. Security personnel can monitor access logs.

6.2 Screen Images

Login Screen

```
-----  
|      Hostel Access System      |  
-----  
| Username: _____          |  
| Password: _____          |  
|                               |  
| [ Login ]                     |  
-----
```

Admin Dashboard

	Admin Dashboard

	1. Register Student
	2. Allocate Room
	3. View Access Logs
	4. Generate Reports

Student Dashboard

	Student Dashboard

	Name: Student Name
	Hostel: Lubombo Hostel
	Room: B12
	[View Status]

6.3 Screen Objects and Actions

Screen Object	Action
Login Button	Authenticates user
Register Button	Registers student
Allocate Room Button	Assigns room
Generate Report Button	Produces hostel reports

Screen Object**Action**

Logout Button

Ends user session

7. REQUIREMENTS MATRIX

Requirement ID	Functional Requirement	System Component
----------------	------------------------	------------------

FR1	User login	Authentication Module
FR2	Register student	Student Management Module
FR3	Allocate hostel room	Room Allocation Module
FR4	Monitor hostel access	Access Control Module
FR5	Generate reports	Reporting Module
FR6	Store student records	Database Module

8. APPENDICES

Appendix A: Future Improvements

Future versions of the system may include:

- Biometric authentication
- Mobile application integration
- SMS and email notifications
- Online hostel applications
- Face recognition access control

Appendix B: Assumptions

- All students possess valid university identification.
- The university provides stable internet connectivity.
- Authorized administrators manage the system.

Appendix C: Tools and Technologies

Tool	Purpose
HTML/CSS	User interface development
PHP	Backend development
MySQL	Database management
XAMPP	Local server environment
Visual Studio Code	Code editor

END OF DOCUMENT

